

S100 VDU

New design boasts many sophisticated features

- ☆ Upper and lower case
- ☆ 16 rows of 64 characters
- ☆ Flashing characters
- ☆ Black-on-white characters
- ☆ Graphics
- ☆ S100 plug-in card

A SOFTWARE CONTROLLED VDU (SCVDU) (otherwise known as a memory-mapped VDU) is one in which the memory that is storing the VDU's display is part of the processor's normal main memory. Hence the processor can read or write directly to any location on the screen without having to progress sequentially down the screen as in conventional VDUs.

The advantages of the SCVDU are almost limitless and new applications will keep occurring to you as you use one. For example, the processor can write to this memory as quickly as it can write to any other memory, so a complete screen of information can be output quicker than you can blink. This allows you to continually look at listings of a program as you modify it without waiting hours. If the screen is displaying the output of a game the display can be updated every few milliseconds to provide the effect of movement. It is possible also to just update one portion of the screen and leave the rest static, such as having a time count in one corner of the screen or having a 'Bytes left to load' count when reading from tape. (This is one facility I wouldn't be without.)

Any portion of the screen can be scrolled as desired, maybe three lines scrolling at the bottom with the rest of the page staying fixed, or scroll downwards just for variety. Writing across the screen and down gets dull, why not make the processor write vertically for a change? Because the processor can read back from the screen it is possible

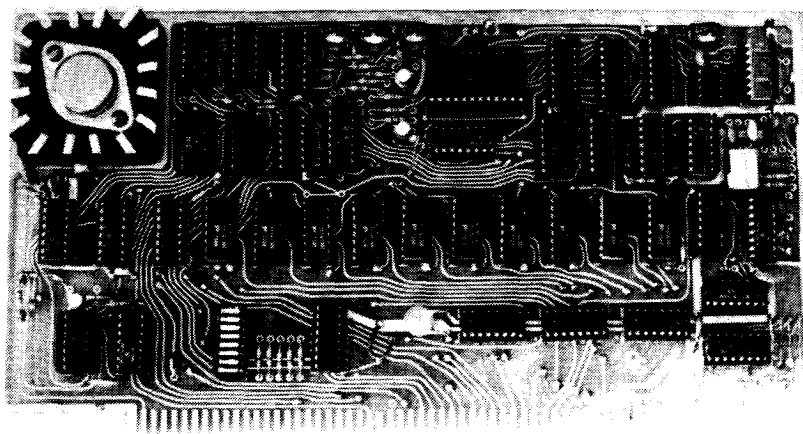


Fig. 1. The ETI 640 VDU board.

to do text editing on the screen including insertion and deletion and then simply read it back without the need to keep a scratch copy elsewhere.

Let your imagination run wild, the display possibilities keep coming.

Screen Format

The characters are displayed as 64 characters per line and 16 lines on the screen. The character generator displays a 7 x 9 dot matrix with appropriate lower case characters (g, j, p etc) descending below the line by 3 dots effectively giving a 7 x 12 display. The font has 128 characters including upper and lower case letters and various mathematical and general symbols (see *Data Sheet*). Other fonts are also available,

in pin compatible ROMs, such as Greek, French, German and Japanese letters.

Flashing Characters

With a SCVDU the processor can make a character(s) flash by writing the character then a blank after a suitable delay and then repeating this cycle. However, this keeps the processor busy when it could be away doing better things and the flash rate will vary depending on how much other work the processor is doing at the time. A much neater alternative is to have the flashing done in hardware and selected by a bit (bit 8) with each character which is how this VDU approaches it. Thus any character on the screen can be selected as flashing or not. The flash

rate is set with a preset control on the board.

Black on White Characters

The area around a character is turned white and the character is seen as a black letter in this area. This is particularly useful to indicate a cursor as it can be backspaced over characters leaving the characters readable. The inverse video function can also be used to emphasize a line. As with the flashing function, Black on White is selected by an extra bit for each character. (Bit 7).

Dinky Graphics

These are low resolution graphics that are useful for large headings or borders, or for simple diagrams. The graphics are formed by taking the area occupied by one character (the same area seen as white around a Black on White character) and breaking it up, two across and four down. (See How it Works). These eight dots can be selected as White or Black by the eight bits that are stored for the character that would have occupied the space. Graphics or normal is selected by another bit for each character.

ave

Memory Organisation

As you can see from the above points, there are quite a few bits stored for each character, ten in fact. This is how they are organised:

- Bits 0-6 Seven bit ASCII code (giving 128 characters)
- Bit 7 Black on white bit
- Bit 8 Flashing bit
- Bit 9 Graphics bit

Storing 10 bits per character presents a problem for an eight bit micro-processor. The way around the problem is to store the first eight bits of information for the 1024 characters on the screen as one kilobyte and then store the extra two bits exactly 1024 addresses above the character that they belong to. Thus the VDU occupies 2 Kilobytes of addresses but the second K only has two bits.

As an example of the above arrangements, if you locate the VDU at F000 HEX then it will occupy from F000-F7FF (2K). The character in the top left hand corner of the screen will be stored at F000 HEX and the bits to make this character flash or be a graphic are stored at F400 HEX. The last character (at the bottom right hand corner) will be stored at F3FF HEX.

S100 Bus Standard

The S100 standard is a 100 pin bus standard that was introduced by Altair

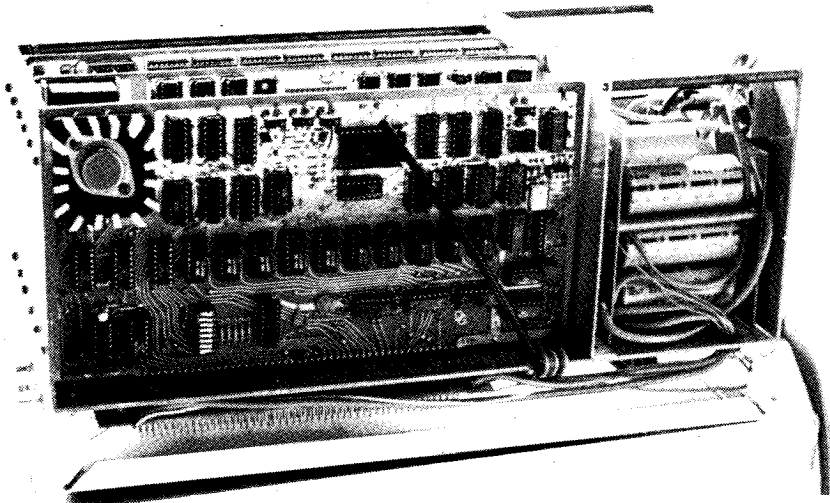


Fig. 2. The ETI 640 VDU is designed to plug into any S100 microcomputer system, as is shown here.

and has been widely accepted in the USA. S100 was originally designed around the 8080 processor but can be adapted to virtually any processor. If your system is not S100 oriented all you need do is plug the VDU into a 100 pin socket and wire the appropriate signals out of your processor. None of the signals used is specialised and should be available on any processor or at least derivable with two or three gates.

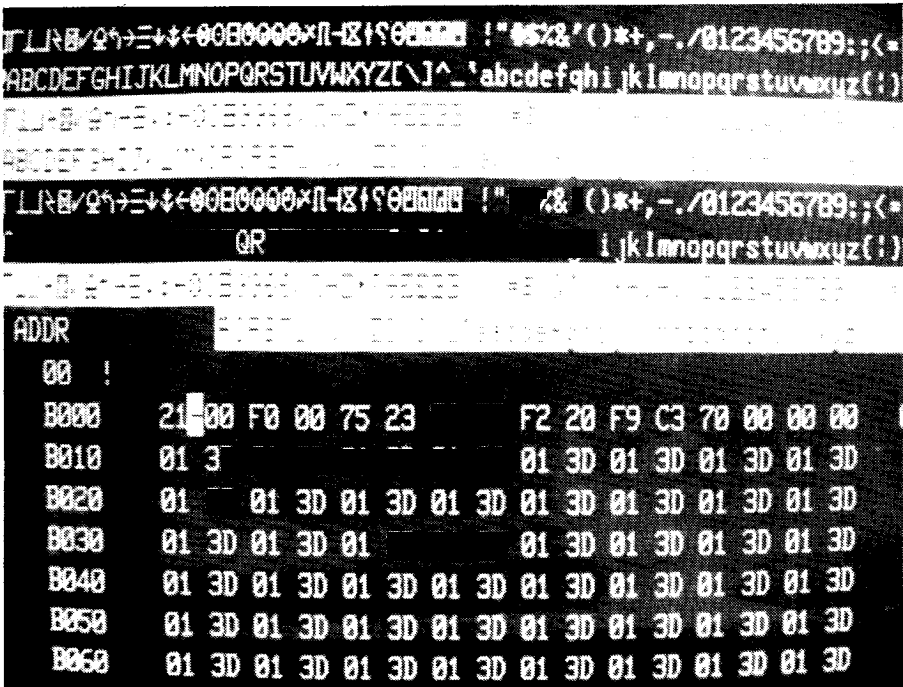


Fig. 3. This photograph is taken from the screen of a monitor and shows the character set of the 640 VDU.

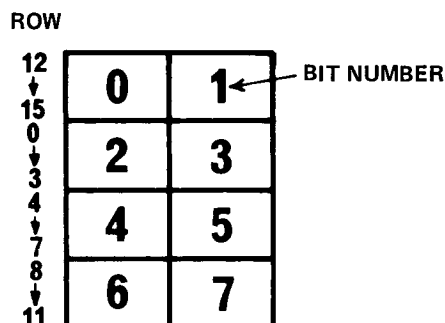


Fig. 4. Each character space is broken up for the creation of graphic characters.

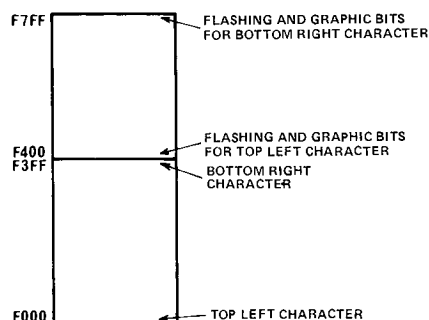
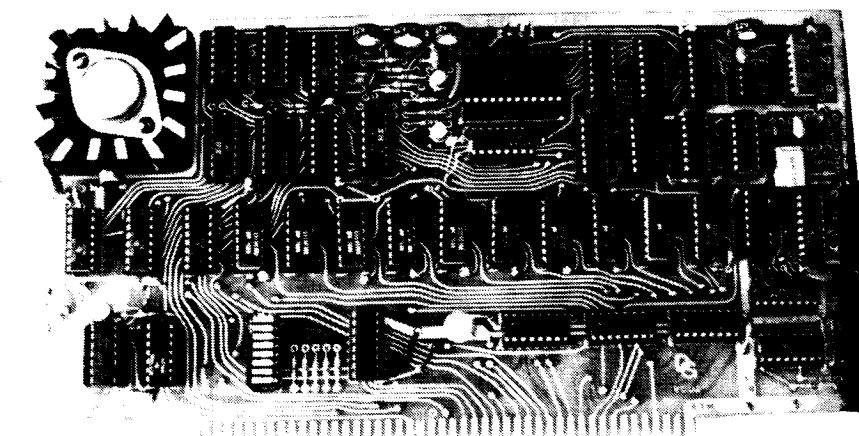


Fig 5. The VDU actually looks to the processor like 2K of memory.



Synchronising Pulse Generator

The start of the chain is the 12MHz crystal oscillator (IC9) which is divided down to 15,625 Hz and 50 Hz (by IC's 38, 39, and 40). The 15,625 Hz signal triggers a 5 μ S monostable (IC 41A) to provide line synchronising pulses and the 50 Hz signal triggers a 300 μ S mono (IC41B) for field synchronising pulses. These synchronising pulses are added to the final video signal before it leaves the board to give composite video out as well as providing a timing reference for the rest of the VDU.

Character Generation

It is not intended to give a complete explanation of how characters are generated on a screen as this has been explained in previous VDU articles and the basic principles used here are the same. The characters are displayed on a matrix 7 dots across by 9 high. The character generator ROM (IC4, MCM6574) outputs the 7 bits across each character in parallel as it is told what character is being constructed (on pins 15, 16, 12, 11, 9, 8, 4,) in ASCII code and which row of the character is selected (on pins 21, 22, 23, 24).

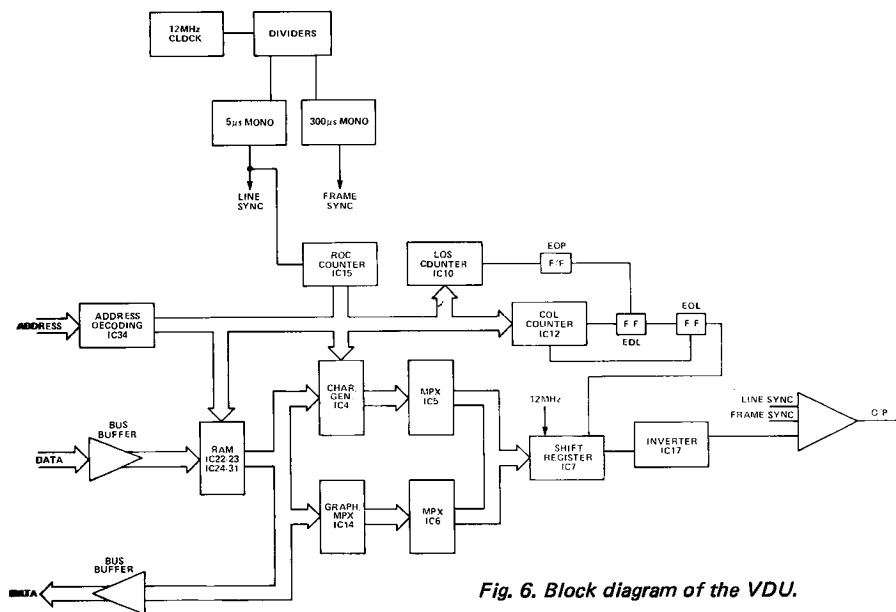
IC 5 and IC 6 are Quad 2 in 1 out multiplexers which simply act as an eight pole two position switch. They connect the output of the character generator ROM (IC4) or the output of the graphics multiplexer (IC14) through to the parallel inputs of the 8 bit shift register (SR) (IC7). Ignoring the graphics mode for a moment, the output of the ROM is connected through to the SR and the eighth bit is tied low (pin 14, IC5) to give black; this is the space between each character. Once

the SR is loaded with this information and is clocking it out at the right rate (12 MHz) the RAMs and ROM are already looking up the information for the next character as both the RAMs and the ROM have a noticeable access time to consider.

The data out of the SR is essentially the information that goes to the screen, although there is some gating after it to invert the data for flashing characters and inverse video, as well as blanking circuitry which will be discussed later.

Counters

The above combination of RAM, ROM and SR will happily produce characters all over the screen provided the RAMs are given information about which character is being produced (ie character on the line, and line on the screen) and the ROM is told which row of the character is being produced, so there are three sets of counters which keep track of this information. The first set (ICs 11 and 12) count 0-63 for character on the line (COL). This counter is incremented by the same pulse that loads the SR (from IC13, pin11) at the end of each character. When this set of counters reaches zero (64) it sets the first 'end of line' flip flop (IC2B). IC2B enables the second 'end of line' flip flop to set after the 64th character has been clocked out by the SR. This combination of two flip flops is necessary to provide a one character delay, otherwise the screen would be blanked as soon as the counters reached zero and the 64th character would be lost. Aha! When the second 'end of line' FF has set it inhibits any further clocking



of the SR and COL counters and forces the screen to black (via the diode to IC16, pin12) until the beginning of the next line. The clearing of the next FFs is discussed under 'Positioning'.

The second counter that counts the row of the character (IC15) is incremented every time the 'end of line' FF sets. One of the confusing parts of the circuit is the fact that this counter actually counts 12, 13, 14, 15, 0, 1, 2, 10, 11 for each character. The ROM outputs blanks for rows 12, 13, 14, 15 so this puts the four lines of blanks above the character instead of below. The result of this is that each character is centred in the character position for Black on White characters, rather than being hard up against the top of the white area. The effect of the row counter counting in this sequence is achieved by incrementing the third counter (the 'line on screen', LOS) when the row counter reaches 12. (IC 16 pins 9, 10 gates C, D of Row Counter).

The LOS counter counts 0-15 for the 16 lines of characters on the screen and when it overflows to zero it sets the 'end of page' flip flop (IC2A). This will occur at the end of a line, as that is when the row counter is incremented, that then increments the LOS counter, so the 'end of line' flip flops will both be set. The end of the page' flip flop simply stops the 'end of line' FFs being cleared until the top of the next page.

Positioning

The position of the characters on the screen is determined by the time after the synchronising pulses that the EOL and EOP FFs are cleared. These two times

(Vertical and Horizontal) are set by two monostables (IC3A and IC3B respectively) which are triggered off the leading edges of the vertical and horizontal synchronising pulses respectively. As each of these monos drop off they cause a negative going pulse via a capacitor to the reset pins of the appropriate FF. The EOP FF holds a reset on the horizontal positioning mono.

The final counter which wasn't mentioned earlier is the counter (IC 13) which counts the number of bits that the SR has clocked out to provide a load pulse for the next character. This counter is clocked at the same rate as the SR (12 MHz) and counts to 9 before resetting to zero, so the SR clocks out the eight bits it has been loaded with plus one more which will be whatever was on the Serial In Pin of the SR (pin 10) when it clocked out the first bit. This is arranged to be a zero (black) when making characters or whatever is appropriate when making graphics.

Graphics

The graphics circuitry only needs to be told which row of the line is being made to select which two bits to feed to the SR (via the graphics/character multiplexers). The same two bits are selected for four rows so only the two highest bits of the row counter are used by the graphics multiplexers (IC 14).

Black on White Characters (Inverse Video)

The 'black on white' (BOW) bit (bit 7) is latched by a 'D' Flip Flop (IC18A) at the same time the SR is loaded. This bit is forced to normal when a graphic is being made. (IC16A) as there is obviously no

point in inverting a graphic and the eighth bit (bit 7) is used by the graphic anyway. The output of the FF controls an exclusive OR gate which acts as a switchable inverter, if a '1' is held on the 'control' input (pin 10) the data on the other pin will be inverted at the output.

The XOR is in the output of the SR so when it is switched to the invert mode it simply inverts the bit stream.

Flashing

The flashing bit (bit 8 – 1st bit 2nd K) is latched in a 'D' FF (IC 18B) the same as the BOW bit described above. When a '1' is stored in the FF it forces the output to black. The flashing is generated by an oscillator (IC8) clearing the FF down at the flash rate.

On The Bus Side Of The Board

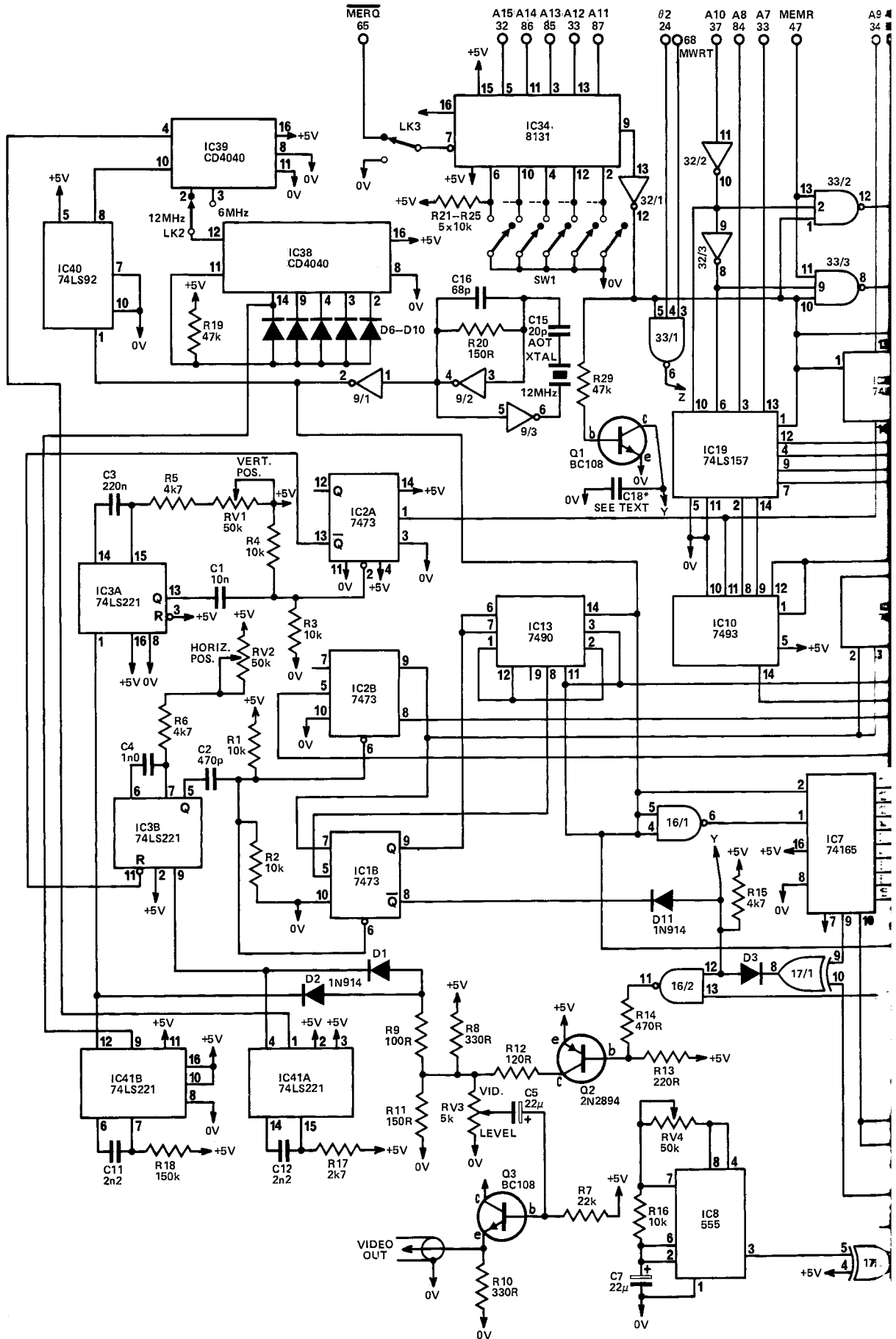
To the processor, the VDU looks just like normal memory with the exception that the upper 1 Kilobyte only has 2 bits. Thus the circuitry involved here is fairly typical of any memory board, the notable difference is that the lower 10 address bits to the RAMs go via multiplexers so that the VDU counters can scan the RAMs when the processor is not accessing them.

The incoming data (off the bus) is buffered by IC35 and part of IC37 before going to the RAMs. The outgoing data is buffered by IC36 and IC37 which are enabled whenever the board is addressed for a read cycle.

IC34 is a comparator that looks for a match between the DIP switches and the high order address bits on the bus. When it finds a match (pin 9 = '0') it means the board is being addressed and the RAMs are taken away from the VDU temporarily. During this period the screen is forced to black (by the transistor off IC34's output otherwise a series of white flickers would be seen on the screen).

Output Circuit

The output circuit combines the digital (5 V) signal representing the black and white information with the vertical and horizontal synchronising signals to give an analogue video signal which is the output signal from the VDU board. The shift register is being clocked at 12 MHz, so the worst case output signal would be one bit on and one bit off which represents a 6 MHz video signal. The transistor used in the output stage is critical as it needs a very fast rise time. A domestic TV receiver has a sound trap at 5 MHz after the IF strip so the loss of quality by modulating the VDU signal would be considerable. For this reason it is suggested that the TV set be modified to accept video in or a proper video monitor be used. A lot of sets on the market today have video in and out facilities for VTR's already fitted.



In order to minimise both the size and complexity of the circuit diagram, the two blocks of RAM have been represented as one box on the drawing. On the top edge of the box, it can be seen that each DO line goes, via a buffer, to pin 11 of a different 2102 RAM IC, and likewise, each of the DI lines comes from pin 12 of a different IC (right edge). The address lines go to all of the IC's in parallel. Note also that, in S100 parlance, the DO (Data Out) lines are outputs from the CPU (hence inputs to the VDU) and the DI (Data In) lines are CPU inputs, (hence outputs from the VDU). Further information on the S100 bus signals can be found in our September 1977 issue.

MERQ (pin 65) is not a regular S100 signal, but was included on the prototype as a means of avoiding some potential problems with the VDU responding to 'mirrored' I/O addresses put out on the upper eight bits of the 8080 address bus. In practice, this is not a problem, and pin 7 of IC34 is normally linked to ground.

Two other links on the board (above IC39 and above IC10) allow 32 character/line operation and operation with a 6 MHz clock. However, the 6 MHz clock modification is not fully implemented, being a 'hangover' from a previous prototype design - nonetheless, we have left it in for the benefit of those incorrigible individuals who must modify their projects.

The data sheet for the MCM6574 reveals that this device requires +5V, +12V and -3V supplies. Where, I hear you ask, does the -3V come from? The answer lies in IC9, a 7404, three gates of which are connected as an oscillator which pumps charge into the D12-D15, C9, 10 and C17 network to produce a negative supply.

Construction

The ETI 640 is not very difficult to assemble since there is no mechanical construction involved. However, it is a very complex board, and several points should be noted in order to ease construction. Firstly, the board was designed to be through-hole plated to simplify construction and improve reliability. Although it is possible to build the VDU on a double sided board by soldering links through the board, we strongly recommend use of the through-hole-plated board. Our prototypes were constructed the hard way, by soldering both sides of the board, and believe me, it's no fun! Although the through-hole-plated board is more expensive, we believe the extra money is well worth spending.

The boards will be available shortly from Applied Technology. For all you diehards who are intent on doing it the hard way, although PCB patterns are not printed in the magazine, they are available from us. Send a large stamped self-addressed envelope to 'VDU PCB's', ETI Magazine, 15 Boundary Street, Rushcutters Bay, NSW 2011 and we shall send the patterns back.

If you are using your own printed circuit board, you should start by soldering links through the board in positions which will later be covered by IC's - i.e. IC4, IC11 and IC16. It is virtually impossible to solder the other links in until most of the other components are in place, so these will have to wait until later, when they can be inserted before the IC's.

Construction should commence with the insertion and soldering of all IC sockets (if used), SW1 and the links. Care should be taken with the orientation of sockets, and later, IC's, since they are not all the same way up. All

PARTS LIST - ETI 640

Resistors –		all ¼W, 5%
R1 - R4	10k	
R5,6	4k7	
R7	22k	
R8	330R	
R9	100R	
R10	330R	
R11	150R (see note)	
R12	120R	
R13	220R	
R14	470R	
R15	4k7	
R16	10k	
R17	2k7	
R18	150k	
R19	47k	
R20	150R	
R21 - R25	10k	
R26	330R	
R27	1k	
R28	4k7	
Capacitors		
C1	10n	
C2	470p	
C3	220n	
C4	1n0	
C5	22µ tantalum	
C6	10n	
C7	22µ tantalum	
C8	82p	
C9,10	1n0	
C11,12	2n2	
C13	22µ tantalum	
C14	1µ	
C15	20p	
C16	68p	
C17	100n	
C18	10p (see note)	
C19,20	22µ tantalum	
C21 - C28	10n (see text)	
C29 - C39	10n (see text)	
Potentiometers		
RV1,2	50k trimpot	
RV3	10k trimpot	
RV4	50k trimpot	
Semiconductors		
IC1,2	7473	
IC3	74LS221	
IC4	MCM6574	
IC5,6	74LS157	
IC7	74LS165	
IC8	NE555	
IC9	7404	
IC10 - IC12	7493	
IC13	7490	
IC14	74LS153	
IC15	7493	
IC16	7400	
IC17	7486	
IC18	7474	
IC19 - IC21	74LS157	
IC22 - IC31	21L02-1	
IC32	74LS04	
IC33	74LS10	
IC34	8131	
IC35 - IC37	8097	
IC38,39	CD4040	
IC40	74LS92	
IC41	74LS221	
IC42	LM309K	
O1	8C108	
O2	2N2894 or 2N4258	
O3	BC108	
D1 - D11	1N914	
D12 - D15	OA91	
ZD1	12 V, 400 mW zener	
Miscellaneous		
X1	12 MHz, HC18/U crystal	
SW1	8-way BOSS or Utilux DIL switch	
PCB	ETI 640 (see text)	
Heatsink for IC42, solder, wire, etc.		

Capacitors

C1	10n
C2	470p
C3	220n
C4	1n0
C5	22µ tantalum
C6	10n
C7	22µ tantalum
C8	82p
C9,10	1n0
C11,12	2n2
C13	22µ tantalum
C14	1µ
C15	20p
C16	68p
C17	100n
C18	10p (see note)
C19,20	22µ tantalum
C21 - C28	10n (see text)
C29 - C39	10n (see text)

the resistors should be inserted next, followed by capacitors C1-C28, except for C13 which is later mounted on the back of the board. The diodes and transistors can now be soldered in, followed by the potentiometers, and IC42 with its heatsink. Now the board can be flipped over, and C13 soldered onto the back of the board, followed by C29-C39. These 10 nF bypass capacitors mount on the back of the board, across the supply pins of IC's 2, 3, 5, 6, 9, 11, 12, 15, 16, 17 and 18.

At this point, before inserting any other IC's, it is wise to apply power to the board and check for the correct voltage at the regulator output and also on pin 3 of IC4 (which is not yet inserted). If these points are at +5V and +12V respectively, then proceed to either plug in or solder in the remaining IC's, taking care with orientation, and finally solder the crystal in, being careful not to apply excess heat.

This concludes construction of the VDU, and all you have to do now is work up the courage to switch on!

How to Fault Find

Although it would be possible to assemble this project and have it work first off, the number of components and solder joints and the tendencies of Mr

Murphy (and his famous law) make it quite probable that something won't be right, unless you are extra careful. So with this in mind we have outlined a logical procedure for debugging the VDU.

The first check is one that applies to all projects: a thorough inspection of the board and soldering, looking for bridged tracks or dry joints. Included in this inspection should be a check of the orientation of all components, especially IC's and tantalum capacitors, and the values of all discrete components.

Having ascertained that nothing looks amiss with the board the next step is to look at it at a more technical level. First check the voltage supplies to the board, the input to the voltage regulator (IC42) should have between +8V and +10V on it depending on your power supply and there should be about +16V on the bottom end of R26 (Zener dropper).

An easy place to check these two supplies after regulation and the -3V supply, which is generated on board, is on the pins of the character generator ROM (IC4). Pin 1 is (or should be) -3V, pin 2 is +5V, pin 3 is +12V.

Having checked the voltage supplies, the easy part is over and oscilloscope is almost mandatory from here on in. If you do not have a CRO think seriously about where you can acquire access to one.

With the aid of the CRO, the first thing to look for is output from the 12 MHz Master Oscillator (IC9). Pin 2 of IC9 should carry a 12 MHz square wave with an amplitude of at least 3.5V, which will probably look somewhat sinusoidal unless you have a wide bandwidth CRO. If this output is missing check around the oscillator discrete components and the supply pins of IC9.

At this point we are assuming that no output at all is being obtained. If you already have a display of some sort you can skip various bits of this material as you think appropriate.

Sync Pulse Generator

Next check for output from the Sync Pulse Generator. Pin 4 of IC41a should have a waveform like fig. 1 (negative going 5 μ s pulse every 64 μ s) and pin 12 should have a waveform like fig. 2 (negative going 300 μ s pulse every 20 ms). If either of these waveforms is not present check the outputs and inputs of the dual monostable that is

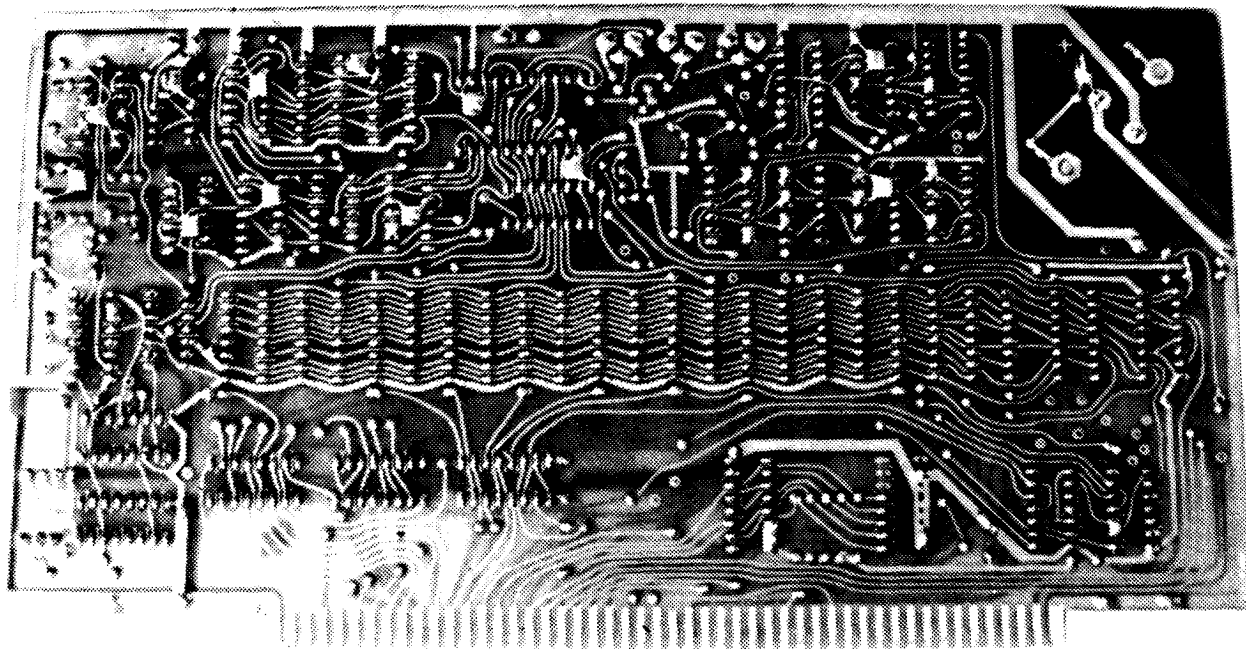


Fig. 4. This photograph shows how the bypass capacitors C29-C39 mount on the back of the board. Since C21-C28, on the front of the board, and C29-C39, on the back, are simply bypass capacitors connected between supply lines and ground, they are omitted from the circuit diagram for clarity.

generating them. Note that actual voltage levels are extremely important in this area of the circuit as MOS chips are interfacing with LS TTL chips. In the prototype one MOS chip was encountered that would not sink its specified output current and hence the TTL chip being driven did not recognize a logic 0 (below 0.8 V).

A 2 MHz signal should be seen at pin 8 of IC40 (divide by six stage from the Master Osc.) then 15,625 kHz (period 64 μ s) should be on pin 4 IC39. IC38 pin 12 should be 31,250 kHz and pin 14 should be 50 Hz (20 ms period).

If any output is not present check the input of the chip concerned, then if this is correct suspect the chip; but don't overlook the possibility of something else holding a short on its output. An easy way of checking for a short is to bend the output pin up (if sockets are used) and measure it completely open circuit.

Before speaking too harshly about any chip remember to check its supply voltage (and earth) and check that it is plugged in the right way round!

Positioning Monostables

The vertical and horizontal positioning monostables (IC3a & IC3b) are triggered by the leading edge of Vertical and Horizontal sync respectively. The output of each of these monos should look like the input with the pulse width being dependent on the associated trippot. The horizontal positioning mono is held reset when the End Of Page (EOP) flip flop is set, so if you are getting the correct output from the Horizontal positioning mono you can skip the next paragraph.

The Q output of the Vertical positioning mono is differentiated by R3, R4, & C1 to give a negative going pulse on the reset input of the EOP flip flop. (IC2a, pin 2). This pulse should look like fig. 3. If this flip flop is not resetting check the discrete components in the differentiator. If the EOP flip flop is being reset correctly then the Horizontal positioning mono has no excuse for not working.

The End Of Line (EOL) flip flops (IC1b, IC2b) are reset by a differentiator similar to the one just described above (R1, R2, C2). The wave form on pin 6 of IC's 1b & 2b should be like fig. 4. The EOL ff's hold the entire counter chain when set, so it is necessary to obtain the correct output from both EOL ff's before venturing any further.

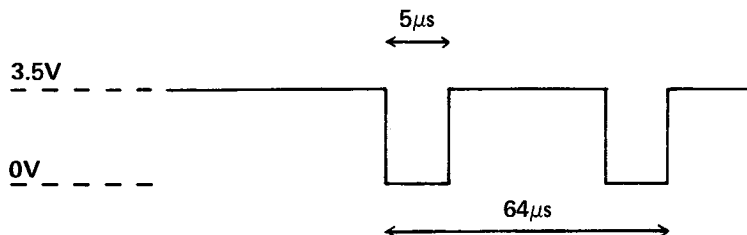


Fig. 1. Waveform at pin 1 of IC41.

Note: logic '1' voltage could be between 3 volts and 5 volts

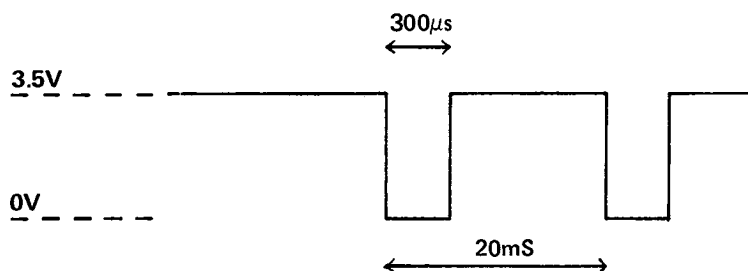


Fig. 2. Waveform at pin 12 of IC41.

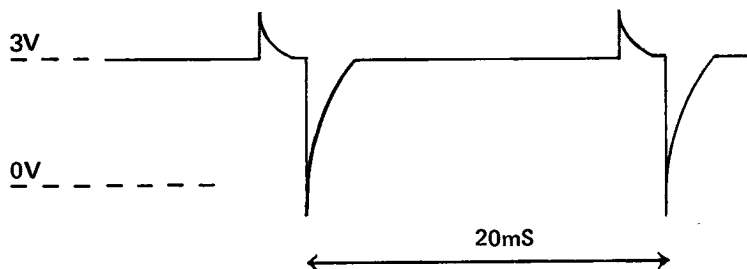


Fig. 3. This waveform should be seen on the reset input of IC2A (pin 2).

Counter Chain

The first counter is the one which counts the bits across each character (IC13, 7490). This counter is wired to reset to zero when it reaches a count of nine and is inhibited from counting (by the EOL ff) by having a reset nine (pin 6) held on it, which takes priority over reset zero. This counter is clocked at 12 MHz and the QD output (fig. 5) on pin 11 is used to load the shift register, clock the next counter (IC12) and clock the flipflops (IC18A and IC18B).

Remember that you will see gaps in the counter waveforms as they are

inhibited from counting for about 13 μ s at the end of each line and also during the vertical blanking period for about 1 ms.

On the outputs of IC12 you should see a normal binary count, counting once for each character (each pulse from QD of IC13). This count should also be seen on the QB and QC outputs of IC11. The first (QA) and last (QD) flipflops in IC11 are not used.

As the QC output of IC11 goes low, (after the 64th pulse into IC12) it will set the first EOL ff (IC2B). (Check that the link, LK1, between IC11 and IC2B

is installed). One character width later, the second EOL ff (IC1B) will set as the OC output of the first counter goes low.

The Row of Character counter (IC15, 7493) counts once every time IC2B sets, so on the Q outputs of IC15 you should see a binary count once every line (64 μ s).

Line of Screen Counter

IC10 is another 4 bit binary counter that counts which line of characters is being produced. It is clocked when IC15 reaches a count of 12 (via IC16/3).

Having ascertained that all the counters are working, the next thing to check is that the count is reaching the RAMs via the address multiplexers. The simplest way to do this is by a look at each address pin of the end RAM (IC31) (pins 1,2,4,5,6,7,8,14,15,16) ensuring that there is a waveform on each pin without worrying what the waveform looks like. Before doing this, however, check the select pin of any of the three multiplexers (IC19, 20, 21 – pin 1), which should all be logic zero. If they are not, the board is being addressed and a check around the board address decoding is required (see below).

Bus Interface

Before looking at the video gating, SR and ROM, it may be desirable to check the interfacing to the bus so that known patterns of characters can be written into the RAM.

Firstly, write '00' Hex to the first location using your processor's monitor program and then read it back. Repeat this process with some other characters, or better still, write a simple program to write, then read, all 256 bit patterns to all locations in the first 1 K of the VDU. If the memory tests all right (i.e. what gets written also gets read) then you can skip the next section. Remember when checking the second K of memory that only the lowest order two bits are stored.

Select an address for the board, e.g. F000 – F7FF, (not F800 – FFFF) and put the processor in a loop reading from the first address of the VDU (e.g. F000). The output of the address decoder (IC34) should be normally high with a pulse going low every 20 μ s or so in a typical system. The link connected to pin 7 of IC34 should be connected to earth.

Pin 12 of IC 33 should look almost identical to the previous waveform. If the program is now altered to read from the last location of the VDU, e.g. F7FF, the same waveform should be seen on IC33 pin 8. Change the program to write repeatedly to the first location

fig. 4.

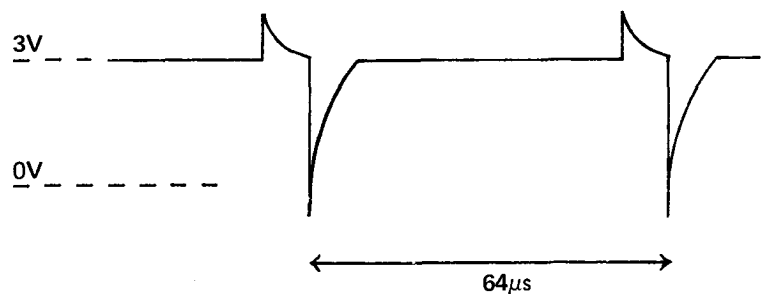


Fig. 4. The waveform on pin 6 of IC's 1B and 2B.

and you should see a similar waveform on IC33 pin 6.

Now fill the screen with an assortment of characters, by writing an incrementing count to each location; for example '00' in F000, '01' in F001, '02' in F002, etc.

Note: this can be done by altering the MOV M,A to a MOV M,L (75H) in the 8080/Z80 page clear routine.

If you put this pattern right through the second K of memory as well, every second character will flash and every second pair will be graphics.

The waveforms around the ROM and shift register are constantly changing, making it virtually impossible to trigger an oscilloscope, but this doesn't matter as the actual waveforms are not important. As long as a changing waveform with suitable amplitude (3.5 – 4 V) is seen on all appropriate pins then all is well.

All the address pins to the ROM should fall into this category. Check pins 4, 8, 9, 11, 12, 15, 16 of IC4. If any pin is found to be held either high or low or if the amplitude is less than 3 V then look for shorts around this pin.

Continue checking all inputs and outputs of the multiplexers (IC's 14, 5, 6) and the inputs of the SR (IC7). On the output of the SR (pin 9) you should see a waveform that begins to resemble a video signal, i.e. a constantly changing waveform with gaps at horizontal and vertical blanking rates.

Inverse Video

The digital information is inverted by IC17 to create black on white characters. If any problems exist in this area check around the D flipflop (IC18A).

The output from the BOW inverter (IC17 pin 8) has the blanking added to it by the diode OR gate (negative logic)

from the second EOL ff IC1B. The output is also forced to black at this point by the transistor whenever the processor accesses the VDU. If this point (IC16 pin 12) is being held low, lift the diode to IC1B and the collector of the transistor in turn to isolate the cause.

Flashing

The signal is forced to black whenever a flashing character is being accessed but not displayed by IC16/2. If there is no output or a character cannot be selected as flashing, check that the 555 (IC8) is producing a 2 Hz square wave on pin 3 and that all signals around the D flipflop appear normal.

Video and Sync Combiner

The only thing that stands between this point and a genuine composite video signal is a handful of discrete components.

A composite video signal is nominally 1 V peak to peak (i.e. from the bottom of sync to the top of white level) and syncs comprise only about 0.3 V of this overall 1 V. It is possible to wind the output level up to about 4 V using the 'video level' trimpot if your monitor needs this much drive.

Conclusion

The approach outlined above is really just a logical approach to faultfinding any digital circuit; start at one end and work to the other until a chip is found that is not producing the correct output for the given input. Normally it is not necessary to go through the entire circuit in this fashion as an educated guess can put the fault in a particular part of the circuit.

Remember, every minute of care taken in assembly is worth hours of debugging. Good luck and good hunting.

CE	F0	00			LDX F000
86	20				LDA A 20
A7	00			LOOP:	STA A X
31					INX
8C	F8	00			CPX F800
26	F8				BNE LOOP
39					RTS

Fig. 5. 6800 screen clear routine. This is not located at any particular address.

B000	21	00	F0	LXI H 'F000'
B003	3E	20		MVI A '20'
B005	77			MOV M,A
B006	23			INX H
B007	7C			MOV A,H
B008	FE	F8		CPI 'F8'
B00A	C2	03	B0	JNZ B003
B00D	C9			RET

Fig. 6. 8080/Z80 screen clear routine, shown assembled to start at B000.

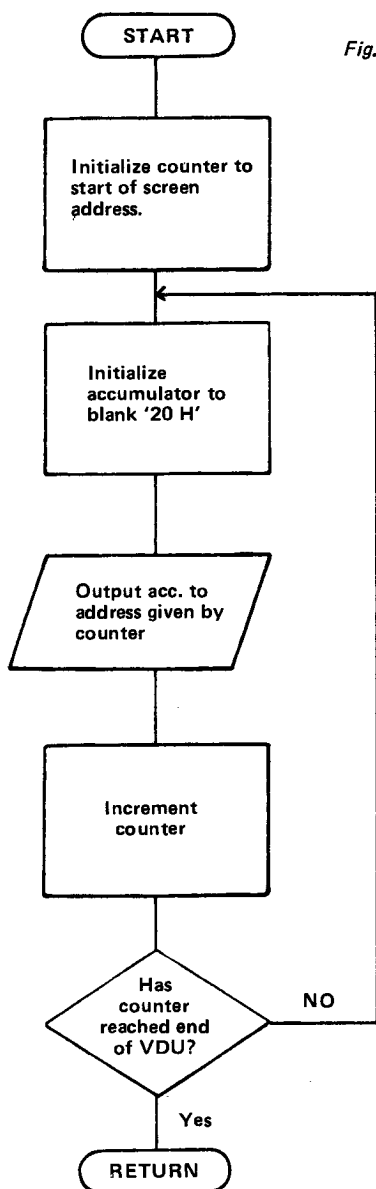


Fig. 7. Flow chart for the screen clear routine.

Software

As with all peripherals it is necessary to have some driver software that knows what the peripheral wants. In this case it is necessary to keep a counter in software of where the next character is to be written and to do decoding for any control functions that it is desired to implement. The normal way this driver would be used is to call it as a subroutine every time it is desired to output a character with the character to be output being passed in a particular register. The driver then looks at the character to determine if it is a control character (such as carriage control, or backspace etc) and if it isn't then it is placed on the screen at the cursor position and the cursor is moved on one place. Since the cursor is something written by the software you can select a variety of cursor symbols. The most useful being to use a black on white character, so when the cursor is sitting over a blank (as it usually is) then it appears as a white block, and when it is backspaced over characters, it is still possible to read the character.

Of course this driver is the normal method of outputting characters, simply operating as a 'glass Teletype' with no programming complications. If you wish to use any of the other facilities of the VDU you simply don't call this driver and access the VDU directly as memory.

A program written for some special application, such as a Radio Teletype (RTTY) substitute can use multiple cursors simply by keeping several counters of output position. Thus one area of the screen could be reserved for

incoming messages while another area is outgoing message preparation area with maybe a line at the top of the page giving log information, like call sign, name, and location of station being worked.

The first part of the driver software, which it becomes immediately obvious is necessary, is a screen clear routine, as the VDU displays chaotic conglomeration of characters and graphics when it is first powered up. This routine is very simple as all it needs to do is write blanks all over the screen and set the flashing and graphic bits for each character to zero. The ASCII code for a blank is 20H and the flashing and graphic bits are the two least significant bits, so if the program writes '20H' 1024 times to make the screen blank and then continues to write '20H' another 1024 times to eliminate any graphics or flashing characters then our object has been achieved. The program to do this can be written in a lot less space than it takes to describe it, see the flowchart in figure 7.

The listings given are not minimized completely but are written to agree with the flowchart for clarity. As you can see all the programs refer to the start of the VDU as 'F000H', this is where I have located the VDU in my system so that it is up near the top of memory and out of the way of program space. It is not advisable to put the VDU at 'F800', the very top of memory, as the processor's bus will usually 'float high' when tri-stated and hence attempt to write to the VDU and blank the screen. If the choice of 'F000H' doesn't conflict with your existing memory map I recommend you use this address to facilitate any software interchange.

Software beyond this is best left to the individual constructor, as everyone has different requirements. It is not difficult to write the simple routines necessary to update the cursor and write characters onto the screen, but there are so many processors in use at present that it is of little value to give them for one particular processor.

In the meantime, we are interested in hearing from readers who have written software compatible with the 640 VDU and will consider publishing software in a section similar to Ideas for Experimenters (for payment, of course!). Reader feedback is invaluable and we would like to hear of your experiences, perhaps for publication to aid other readers.